Lab 2. Accessing Table Data, DML Statements, and Transactions

Content

Unlocking sample user accounts

Oracle Database 11g XE comes with *sample database users* such as **HR**, **MDSYS**, and others. Some of the user accounts are, by default, locked. In the rest of the chapters, we will use HR schema objects to test and build applications.

Log on to SQL*Plus as **SYSDBA**, query account status for all users, and unlock the **HR** account as shown below:

```
SQL> connect /as sysdba
SQL> select username, account_status from dba_users;
SQL> alter user hr account unlock;     # Unlock the locked account
SQL> alter user hr identified by hr;     # Open the expired account
```

Or:

```
SQL> alter user hr identified by hr account unlock; #Unlock and open
in a single statement
SQL> connect hr/hr
```

Installing SQL Developer

SQL Developer is a graphical tool that enables us to interact with an Oracle database. Using SQL Developer, we can query, create/modify/drop database objects, run SQL statements, write PL/SQL stored procedures, and more.

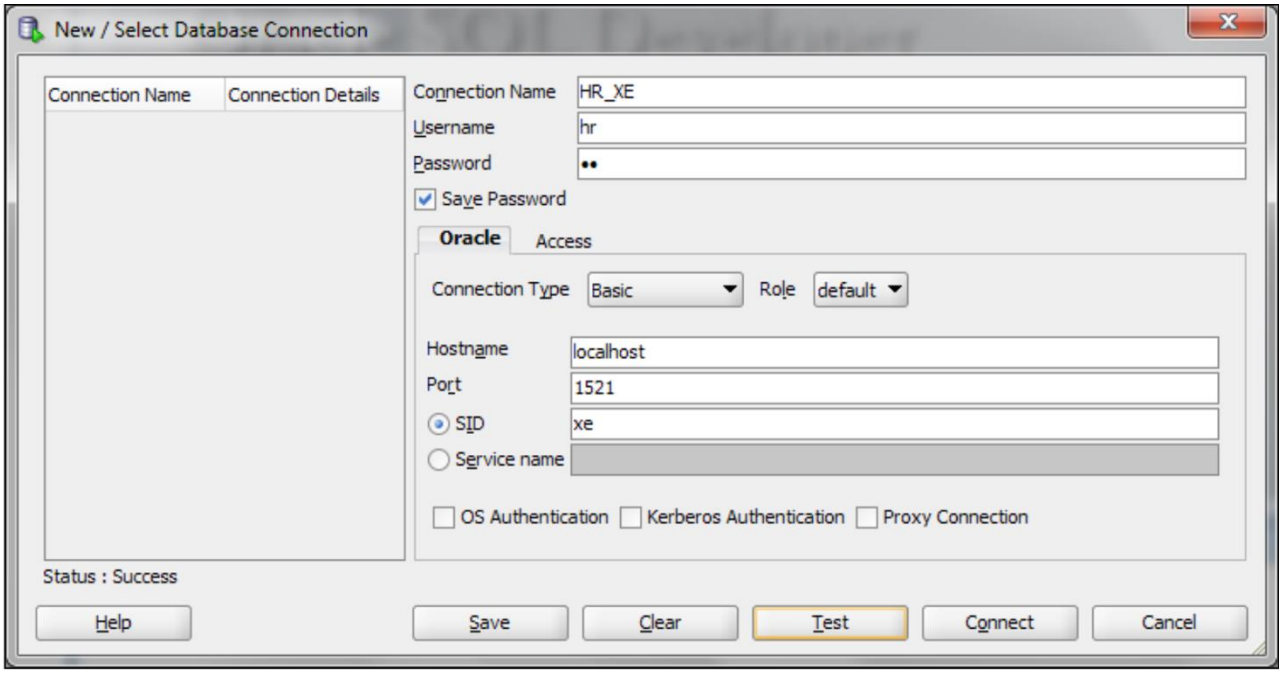
SQL Developer is a separate tool not bundled with Oracle Database 11g XE. SQL Developer is free to download. We can download SQL Developer by following this link: `http://www.oracle.com/technetwork/developer-tools/sql-developer/`.

The following is the procedure to install in a Windows environment:

1. Unzip the **`sqldeveloper-xxx.zip`** to a folder.

2. Navigate to the new folder **sqldeveloper** created by the ZIP file and double-click on the **`sqldeveloper.exe`** file.

Connecting SQL Developer to Oracle Database 11g XE



About DUAL and TAB tables

DUAL is a **SYS**-owned view. It is usually used to return values from *stored functions*, *sequence values*, etc. It is recommended not to drop or perform any DML operations against a DUAL table. The following is an example of fetching the current date using the DUAL table:

```
SQL> SELECT sysdate FROM dual;

SYSDATE
---------
10-MAY-12

SQL>
```

TAB is a **SYS**-owned view used to list tables and views in a table. The following is a sample query to list all tables/views in the current schema:

```
SQL>  select * from tab;

TNAME                          TABTYPE  CLUSTERID
------------------------------ ------- ----------
DEPT                           TABLE
EMP                            TABLE
```

Currently, the TAB view is not widely used. Instead, the **ALL_TABLES** and **USER_TABLES** views are preferred. ALL_TABLES describes the relational tables *accessible* to the current user, and USER_TABLES describes the relational tables *owned* by the current user.

Writing simple queries

We can execute queries using the SQL*Plus environment or SQL Developer.

To view the structure of a table, we use the DESC command:

```
SQL> DESC employees
```

Let us execute a simple query against the Employees table and fetch a few columns:

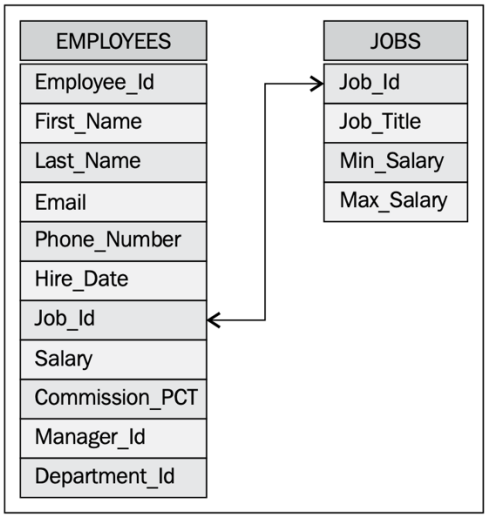
```
SQL> SELECT employee_id, first_name, last_name, job_id FROM employees;
```

We can restrict the results returned by the query using the WHERE clause as shown below:

```
SQL> SELECT employee_id, first_name, last_name, job_id FROM employees
            WHERE salary < 2500;
```

Selecting data from multiple tables

Suppose that we have two tables as below:



Suppose we want to retrieve all employees' IDs, names, and job titles. The necessary information is stored in two tables, and thus we need to join them in the query as follows:

```
SQL>  SELECT e.employee_id, e.first_name, e.last_name, j.job_title
            FROM employees e, jobs j
            WHERE e.job_id = j.job_id;
```

This syntax is called *inner join* in SQL. Beside, we have some other join types in SQL such as outer join (left, right and full outer join). More details on Oracle joins can be found in the following link: https://www.oracletutorial.com/oracle-basics/oracle-joins/.

Exploring common functions

We can use **Oracle Supplied functions** in our queries. We can also create our own functions and use them in queries later. Functions in Oracle can be of type scalar or aggregate. Scalar functions operate on a single row, whereas aggregate functions work on multiple rows. Some popular Oracle built-in functions:

UPPER	RPAD	MAX	TO_NUMBER
LOWER	LPAD	MIN	TO_DATE
CONCAT or ("||")	SUBSTR	AVG	
LTRIM	COUNT	ADD_MONTHS	
RTRIM	SUM	TO_CHAR	

More Oracle DB built-in functions can be found here:

https://docs.oracle.com/javadb/10.8.3.0/ref/rrefsqlj29026.html

https://www.tutorialspoint.com/functions-in-oracle-dbms

Transaction control statements

A transaction is a sequence of one or more SQL statements treated as one unit. Either all of the statements are performed, or none of them are performed. There are two main Transaction Control Statements (TCS), namely COMMIT and ROLLBACK:

- COMMIT: When we COMMIT a transaction, it means that Oracle has made the change permanent in the database
- ROLLBACK: When we ROLLBACK a transaction, all the changes performed since the previous COMMIT/ROLLBACK are all erased

DML statements

Data Manipulation Language (DML) statements are used to manipulate data in existing tables. INSERT, UPDATE, and DELETE are examples of DML statements. We use INSERT to add a new record to the table, UPDATE to modify one or more columns of a table, and DELETE to remove a record from the table.

The following is an example of an INSERT statement. We insert a new record in the regions table of the HR schema:

```
SQL> INSERT INTO regions VALUES (5, 'Australia');
1 row created
SQL> COMMIT;
Commit complete.
SQL>
```

An example of the UPDATE statement is shown next, where we modify the value of `region_name` from Australia to Aus and NZ. This is done as follows:

```
SQL> UPDATE regions SET region_name='Aus and NZ' Where region_id=5;
1 row updated.
SQL> COMMIT;
Commit complete.
SQL>
```

The following is an example of the DELETE statement. We remove the newly added record from the regions table as follows:

```
SQL> DELETE FROM regions, where region_id = 5;
1 row deleted.
SQL> COMMIT;
Commit complete.
SQL>
```

To make the changes permanent, we have to commit the transaction.

Exercises

1. Unlock the HR account and do the examples in the theory sections.
2. Write SQL queries to:
   a. Display first name and experience of the employees.
   b. Display the length of first name for employees where last name contain character 'b' after 3rd position.
   c. Display employees who joined in the current year.
   d. Display job ID for jobs with average salary more than 10,000.
   e. Display the first name, last name, salary and job grade for all employees.
   f. Display the first name, last name, department number and department name for all employees for departments 80 or 40.
   g. Display the job title, department name, full name (first and last name) of employee and starting date for all the jobs which started on or after 1st January, 1993 and ending with on or before 31 August, 1997.
   h. Display the department name and number of employees in each of the department.
   i. Display years in which more than 10 employees joined.
   j. Display job ID of jobs that were done by more than 3 employees for more than 100 days.
   k. Change job ID of the employee 110 to IT_PROG if the employee belongs to department 10 and the existing job ID does not start with IT.